

STUDY GUIDE

Module 2

Advanced JavaScript for React Development

Table of Contents

Table of Contents	2
1. ES6+ Fundamentals, Scoping & Syntax	5
1.1 Variable Scope Mechanics	5
1.1.1 Memory Lifecycle & Temporal Dead Zone (TDZ)	5
1.1.2 Block Scope (let, const) vs. Function Scope (var)	5
1.1.3 Reassignment vs. Immutability in Reference Variables	6
1.2 Operators & Expression Evaluation	6
1.2.1 The Conditional (Ternary) Operator for Inline Layout Directives	6
1.2.2 Logical AND (&&) Short-Circuit Evaluation Core Quirks	6
1.2.3 Nullish Coalescing (??) vs. Logical OR () with Falsy Values	7
1.3 Statements, Loops, & Interpolation	7
1.3.1 Declarative Expression Statements vs. Imperative Control Flow	7
1.3.2 Template Literals: String Interpolation, Multi-line Formatting, and Tagged Templates	8
2. Functional Programming Pillars	9
2.1 Functional Syntax & Lexical Scopes	9
2.1.1 Named and Anonymous Functions	9
2.1.2 Arrow Functions	9
2.2 Advanced Functional Mechanisms	10
2.2.1 Closures	10
2.2.2 Higher-Order Functions (HOFs)	10
2.2.3 Callback Function	11
3. Advanced String, Array, Object Methods	12
3.1 String Manipulations	12
3.1.1 Search Methods: .includes(), .startsWith(), .endsWith(), indexOf()	12
3.1.2 Modification Methods: .trim(), .replace(), .replaceAll(), .split()	12
3.1.3 Substring Extraction Techniques .slice(), .substring(), .substr()	12
3.1.4 Case Conversion .toUpperCase() .toLowerCase()	13
3.1.5 String Composition & Concatenation .concat(), .repeat(), .padStart(), .padEnd()	13
3.2 Array Manipulations	13
3.2.1 High-Performance Array Traversal .forEach(), .map(), .flat(), .flatMap()	13
3.2.2 Structural Subsetting: .filter() and .slice(), .concat()	14
3.2.3 Multi-Dimensional Aggregation & Accumulation: .reduce()	14
3.2.4 Search & Location Utilities: .find(), .findIndex(), .some(), .every()	14

3.2.5 Element Insertion & Removal .push(), .pop(), .shift(), .unshift()	15
3.2.6 String Conversion .join(), .toString()	15
3.3 Object Manipulations.....	15
3.3.1 Object and Array Destructuring: Default Values, Aliasing, and Nested Extraction	15
3.3.2 The Spread Operator (...): Shallow Copying and Immutability Safeguards	16
3.3.3 The Rest Parameter (...): Variable Argument Capture and Partial Extraction	16
3.4 Structural Copiers Object Manipulations.....	17
3.4.1 Creating Objects, Property Access, Adding, Updating, and Deleting Properties, Dynamic Property Names	17
3.4.2 Object Traversal & Built-in Utilities: Object.keys(), Object.values(), Object.entries(), Object.assign(), Object.freeze(), Object.seal(), Object.hasOwn().....	17
3.4.3 Creating & Structuring Arrays of Objects.....	18
4. Numbers, Math, & Dates Methods.....	19
4.1 Number Methods.....	19
4.1.1 Number.parseInt(), Number.parseFloat(), Number.isNaN(), .toFixed()	19
4.2 Math Methods	19
4.2.1 .floor(), .ceil(), .round(), .trunc().....	19
4.2.2 Comparative Selection: .min(), .max(), and Randomization Patterns (.random()).....	19
4.3 Temporal Engineering via the Date Methods	20
4.3.1 Epoch Timestamps, Instantiation, and Timezone Offsets	20
4.3.2 ISO Formatting, Localization Routines	20
5. Asynchronous Architecture, API Handling, & The Event Loop	21
5.1 Async Foundations & The Runtime Infrastructure	21
5.1.1 The Event Loop Engine: Concurrency Model Architecture	21
5.1.2 Call Stack vs. Web APIs Background Execution Framework	21
5.1.3 Microtask Queue (Promises) vs. Macrotask Queue (setTimeout) Priority Routing.....	21
5.2 Promise State Topologies.....	22
5.2.1 Lifecycle Phases: Pending, Fulfilled, Rejected.....	22
5.2.2 Execution Control: .then(), .catch(), .finally().....	22
5.2.3 Concurrent Aggregation Engines: Promise.all(), Promise.allSettled(), Promise.race().....	22
5.3 Streamlined Async Control Flows	23
5.3.1 async/await Syntactic Sugar Engine Mechanics	23
5.3.2 Robust Network Exception Management via try-catch Blocks.....	23
5.3.3 HTTP Data Ingestion: Fetch API Pipeline, Response Validation, and Parsing Strategies	24
6. Browser Storage APIs & DOM Architecture.....	25

6.1 Client-Side Persistent Engines.....	25
6.1.1 localStorage vs. sessionStorage: Domain Scopes and Retention Lifecycles	25
6.1.2 Serialization Mechanisms: Data Transformation with JSON.stringify() and JSON.parse()	25
6.2 Document Object Model (DOM) Mechanics.....	25
6.2.1 The Browser Node Tree Structure and Rendering Layout Pipeline	25
6.2.2 Imperative Element Interception and Mutation Patterns	26
6.2.3 Event Delegation, Event Bubbling, Capturing, and Performance Optimization	26
8. TypeScript Integration & Methods	27
8.1 Type System Foundations	27
8.1.1 Primitives (string, number, boolean, null, undefined) vs. any/unknown	27
8.1.2 Structure Typing: Defining Contracts via type Aliases vs. interface Systems	27
8.1.3 Flexible Types: Union Types, Intersection Types	28
8.2 TypeScript Functional & Transformation Methods	28
8.2.1 Strongly Typing Function Arguments, Return Structures, and Async Promises	28
8.2.2 Generics (<T>): Creating Reusable, Type-Safe Data Contracts	29
8.2.3 Advanced Utility Mappings: Partial<T>, Omit<T, K>, Pick<T, K>, and Readonly<T>.....	29
8.3 Object-Oriented Programming (OOP).....	29
8.3.1 Class Fundamentals: Class Declaration, Object Creation, Constructors, Properties, Methods.....	29
8.3.2 Access Modifiers: public, private, protected, readonly	30
8.3.3 Encapsulation: Data Hiding, Getter Methods, Setter Methods, Access Control	31
8.3.4 Inheritance: extends, Base Class, Derived Class, Method Overriding, super	31
8.3.5 Polymorphism: Runtime Polymorphism, Method Overriding	32
8.3.6 Abstraction: Abstract Classes, Abstract Methods.....	32
8.3.7 Interfaces in OOP: Implementing Interfaces, Multiple Interface Implementation, Interface Extension.....	33
8.3.8 Static Members: Static Properties, Static Methods	33

1. ES6+ Fundamentals, Scoping & Syntax

1.1 Variable Scope Mechanics

1.1.1 Memory Lifecycle & Temporal Dead Zone (TDZ)

Before your code actually runs, JavaScript quickly scans it and reserves memory for your variables. This is called hoisting.

var variables are hoisted AND set to undefined immediately, so you can (accidentally) use them before their line of code. let and const are also hoisted, but they stay in a locked state called the Temporal Dead Zone (TDZ) until their declaration line runs. Touching them before that throws an error.

Example Syntax

```
console.log(score); // undefined -> var is hoisted and set to undefined
var score = 10;

console.log(age); // ReferenceError: Cannot access age before initialization
let age = 25;
```

Beginner Tip: Always declare your variables at the very top of the block. This avoids TDZ errors completely.

1.1.2 Block Scope (let, const) vs. Function Scope (var)

var is function scoped, meaning it exists everywhere inside the function it was created in, even inside an if block or a loop.

let and const are block scoped, meaning they only exist inside the nearest pair of curly braces { } where they were written. This makes let and const safer and more predictable, which is why modern JavaScript and React code almost always use them instead of var.

Example Syntax

```
function testVar() {
  if (true) {
    var x = 10;
  }
  console.log(x); // 10 -> still accessible outside the if block
}

function testLet() {
  if (true) {
    let y = 10;
  }
  console.log(y); // ReferenceError: y is not defined
}
```

1.1.3 Reassignment vs. Immutability in Reference Variables

const stops you from pointing the variable to a brand new value. But if that value is an object or an array (called a reference type), you can still change its inner contents. Only reassigning the whole variable is blocked.

This confuses a lot of beginners, so remember: const protects the box, not necessarily everything inside the box.

Example Syntax

```
const age = 25;
age = 26; // TypeError: Assignment to constant variable

const user = { name: 'Alice' };
user.name = 'Bob'; // allowed -> only a property changed
user = { name: 'Carol' }; // TypeError -> cannot reassign the whole object
```

1.2 Operators & Expression Evaluation

1.2.1 The Conditional (Ternary) Operator for Inline Layout Directives

The ternary operator is a compact one-line replacement for if-else. The pattern is: condition ? valueIfTrue : valueIfFalse.

In React, this is the standard way to decide what to show on screen, because you cannot write a full if statement directly inside JSX.

Example Syntax

```
const age = 20;
const message = age >= 18 ? 'You can vote' : 'You are too young to vote';
console.log(message);

// Inside a React component's JSX:
// {isLoggedIn ? <Dashboard /> : <LoginPage />}
```

1.2.2 Logical AND (&&) Short-Circuit Evaluation Core Quirks

The && operator checks the left side first. If it is falsy, JavaScript stops right there and returns that falsy value without even looking at the right side. If the left side is truthy, it moves on and returns the right side.

In React, this pattern is used to show something on screen only when a condition is true.

Example Syntax

```
const isLoggedIn = true;
const result = isLoggedIn && 'Welcome back!';
console.log(result); // 'Welcome back!'
```

```
// React JSX pattern:
// {cartItems.length > 0 && <p>You have items in your cart</p>}

// Watch out! If cartItems.length is 0, this quirk shows up:
// {cartItems.length && <Cart />}
// This actually renders the number 0 on the screen, because
// 0 is falsy but it still gets returned and displayed.
```

1.2.3 Nullish Coalescing (??) vs. Logical OR (||) with Falsy Values

|| replaces the left value whenever it is ANY falsy value: 0, empty string, null, undefined, NaN, or false.

?? only replaces the left value when it is specifically null or undefined. This matters a lot when 0 or an empty string are valid, meaningful values that you do not want to accidentally overwrite.

Example Syntax

```
const score = 0;
console.log(score || 10); // 10 -> wrong here, 0 was a valid score!
console.log(score ?? 10); // 0 -> correct, 0 is kept

const username = '';
console.log(username || 'Guest'); // 'Guest'
console.log(username ?? 'Guest'); // '' -> empty string is kept as-is
```

1.3 Statements, Loops, & Interpolation

1.3.1 Declarative Expression Statements vs. Imperative Control Flow

An expression produces a value, for example 5 + 5 or a ternary operator. A statement, like if, for, or while, performs an action or controls the flow of the program, but it does not produce a usable value.

This is exactly why you can write a ternary directly inside JSX, but you cannot write a plain if statement inside JSX curly braces.

Example Syntax

```
// Expression -> produces a value
const sum = 5 + 5; // 10

// Statement -> controls flow, does not return a usable value
if (sum > 5) {
  console.log('Big number');
}

// This is why JSX allows:      {isTrue ? 'Yes' : 'No'}
// but JSX does NOT allow:     { if (isTrue) { 'Yes' } }
```

1.3.2 Template Literals: String Interpolation, Multi-line Formatting, and Tagged Templates

Template literals use backticks (` ) instead of quotes. Inside them, `${ }` lets you insert a variable or expression directly into the string, which is called string interpolation.

They also support multi-line text without needing a special newline character, and 'tagged templates' let a function process the template before it becomes a final string, which is an advanced but powerful feature.

Example Syntax

```
const name = 'Alice';
const age = 25;

// String interpolation
console.log(`Hello, my name is ${name} and I am ${age} years old.`);

// Multi-line string, no \n needed
const message = `Line one
Line two
Line three`;

// Tagged template (advanced) -> a function processes the pieces
function shout(strings, ...values) {
  return strings.reduce((acc, str, i) => acc + str + (values[i] ?
String(values[i]).toUpperCase() : ''), '');
}
console.log(shout`Hello ${name}, you are ${age}`);
```

2. Functional Programming Pillars

2.1 Functional Syntax & Lexical Scopes

2.1.1 Named and Anonymous Functions

A named function has its own identifier. It gets hoisted fully, so it can be called even before its definition appears in the file, and its name shows up in error messages, which makes debugging easier.

An anonymous function has no name of its own. It is usually stored in a variable or passed directly as an argument to another function.

Example Syntax

```
// Named function
function greet(name) {
  return `Hello, ${name}`;
}

// Anonymous function stored in a variable
const greet2 = function (name) {
  return `Hi, ${name}`;
};

// Anonymous function passed directly as an argument
setTimeout(function () {
  console.log('This runs after 1 second');
}, 1000);
```

2.1.2 Arrow Functions

Arrow functions are a shorter way to write functions using `=>`. Unlike regular functions, they do not create their own 'this' value; instead they borrow 'this' from the code around them, which is called lexical scoping.

This makes arrow functions extremely popular for callbacks and React components.

Example Syntax

```
// Regular function
function add(a, b) {
  return a + b;
}

// Arrow function -> same result, shorter, implicit return
const add2 = (a, b) => a + b;

// Arrow function with a full block body
const multiply = (a, b) => {
  const result = a * b;
```

```
    return result;
};

// Single parameter -> parentheses are optional
const square = x => x * x;
```

2.2 Advanced Functional Mechanisms

2.2.1 Closures

A closure happens when an inner function keeps access to variables from its outer function, even after that outer function has already finished running.

Closures are the hidden mechanism behind private data in JavaScript, and they are the foundation React's useState hook is built on.

Example Syntax

```
function createCounter() {
  let count = 0; // this variable is enclosed by the inner function
  return function () {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
// count is private -> it can only change through the returned function
```

2.2.2 Higher-Order Functions (HOFs)

A Higher-Order Function is any function that takes another function as an argument, returns a function, or both.

Array methods like map, filter, and reduce are all Higher-Order Functions, because each of them accepts a function as an argument.

Example Syntax

```
// HOF that takes a function as an argument
function repeatAction(times, action) {
  for (let i = 0; i < times; i++) {
    action(i);
  }
}

repeatAction(3, i => console.log(`Action number ${i}`));
```

```
// HOF that returns a function
function multiplyBy(factor) {
  return function (number) {
    return number * factor;
  };
}
const double = multiplyBy(2);
console.log(double(5)); // 10
```

2.2.3 Callback Function

A callback is simply a function you hand over to another function so it can be called later, usually once some task finishes.

Callbacks are everywhere: timers, event listeners, and (before Promises existed) handling asynchronous work.

Example Syntax

```
function fetchData(callback) {
  console.log('Fetching data...');
  setTimeout(() => {
    callback('Data received!');
  }, 1000);
}

fetchData(function (result) {
  console.log(result); // 'Data received!' after 1 second
});

// Real-world example: an event listener callback
button.addEventListener('click', function () {
  console.log('Button was clicked!');
});
```

3. Advanced String, Array, Object Methods

3.1 String Manipulations

3.1.1 Search Methods: .includes(), .startsWith(), .endsWith(), indexOf()

These methods let you check whether a string contains certain text, or find exactly where that text sits inside the string.

Example Syntax

```
const text = 'Hello, welcome to React';

console.log(text.includes('React')); // true
console.log(text.startsWith('Hello')); // true
console.log(text.endsWith('React')); // true
console.log(text.indexOf('welcome')); // 7 -> position where it starts
console.log(text.indexOf('Vue')); // -1 -> not found
```

3.1.2 Modification Methods: .trim(), .replace(), .replaceAll(), .split()

trim removes extra whitespace from both ends of a string. replace swaps out only the FIRST match. replaceAll swaps out EVERY match. split breaks a string apart into an array using a separator you choose.

Example Syntax

```
const msg = '  Hello World  ';
console.log(msg.trim()); // 'Hello World'

const sentence = 'I like cats. Cats are cute.';
console.log(sentence.replace('Cats', 'Dogs')); // replaces only the first match
console.log(sentence.replaceAll('Cats', 'Dogs')); // replaces every match

const csv = 'apple,banana,orange';
console.log(csv.split(',')); // ['apple', 'banana', 'orange']
```

3.1.3 Substring Extraction Techniques .slice(), .substring(), .substr()

slice pulls out a piece of a string using a start and end position, and it accepts negative numbers to count from the end. substring is similar but treats negative numbers as 0. substr is older and considered deprecated, so avoid it in new code.

Example Syntax

```
const str = 'JavaScript';
console.log(str.slice(0, 4)); // 'Java'
console.log(str.slice(-6)); // 'Script' -> counts 6 characters from the end
console.log(str.substring(0, 4)); // 'Java'
```

```
console.log(str.substr(4, 6)); // 'Script' -> deprecated, avoid using it
```

3.1.4 Case Conversion .toUpperCase() .toLowerCase()

These two methods simply convert an entire string to uppercase or lowercase letters.

Example Syntax

```
const name = 'Alice';  
console.log(name.toUpperCase()); // 'ALICE'  
console.log(name.toLowerCase()); // 'alice'
```

3.1.5 String Composition & Concatenation .concat(), .repeat(), .padStart(), .padEnd()

concat joins strings together, though template literals are usually preferred today. repeat copies a string a set number of times in a row. padStart and padEnd add extra characters to the start or end of a string until it reaches a target length, which is very useful for formatting things like 05 instead of 5.

Example Syntax

```
console.log('Hello'.concat(' ', 'World')); // 'Hello World'  
console.log('ab'.repeat(3)); // 'ababab'  
console.log('5'.padStart(2, '0')); // '05'  
console.log('5'.padEnd(4, '*')); // '5***'
```

3.2 Array Manipulations

3.2.1 High-Performance Array Traversal .forEach(), .map(), .flat(), .flatMap()

forEach loops through an array and runs a function for each item, but it does not give you anything back. map loops through and builds a brand new array with transformed values. flat flattens nested arrays into a single-level array by a chosen depth. flatMap maps and flattens in a single step.

Example Syntax

```
const numbers = [1, 2, 3];  
  
numbers.forEach(n => console.log(n)); // just logs 1, 2, 3, returns nothing  
  
const doubled = numbers.map(n => n * 2);  
console.log(doubled); // [2, 4, 6]  
  
const nested = [1, [2, 3], [4, [5, 6]]];  
console.log(nested.flat()); // [1, 2, 3, 4, [5, 6]] -> only 1 level deep  
console.log(nested.flat(2)); // [1, 2, 3, 4, 5, 6] -> 2 levels deep  
  
console.log([1, 2, 3].flatMap(n => [n, n * 2])); // [1, 2, 2, 4, 3, 6]
```

3.2.2 Structural Subsetting: .filter() and .slice(), .concat()

filter builds a new array containing only the items that pass a test you write. slice copies out a portion of an array without touching the original one. concat joins two or more arrays together into a brand new array.

Example Syntax

```
const nums = [1, 2, 3, 4, 5, 6];
const evens = nums.filter(n => n % 2 === 0);
console.log(evens); // [2, 4, 6]

console.log(nums.slice(1, 3)); // [2, 3]

const arr1 = [1, 2];
const arr2 = [3, 4];
console.log(arr1.concat(arr2)); // [1, 2, 3, 4]
```

3.2.3 Multi-Dimensional Aggregation & Accumulation: .reduce()

reduce walks through an array and boils it down into a single final value, which could be a number, a string, an object, or even a brand new array. You supply a function that updates a running 'accumulator' on every step.

Example Syntax

```
const prices = [10, 20, 30];
const total = prices.reduce((accumulator, current) => accumulator + current, 0);
console.log(total); // 60

// Grouping example -> group names by age
const people = [
  { name: 'A', age: 20 },
  { name: 'B', age: 20 },
  { name: 'C', age: 30 },
];
const grouped = people.reduce((acc, person) => {
  const key = person.age;
  if (!acc[key]) acc[key] = [];
  acc[key].push(person.name);
  return acc;
}, {});
console.log(grouped); // { 20: ['A', 'B'], 30: ['C'] }
```

3.2.4 Search & Location Utilities: .find(), .findIndex(), .some(), .every()

find returns the FIRST item that matches your condition, or undefined if nothing matches. findIndex returns the position of that first match, or -1. some checks whether AT LEAST ONE item passes the test. every checks whether ALL items pass the test.

Example Syntax

```
const users = [
  { id: 1, active: false },
  { id: 2, active: true },
];

console.log(users.find(u => u.active));    // { id: 2, active: true }
console.log(users.findIndex(u => u.active)); // 1
console.log(users.some(u => u.active));    // true
console.log(users.every(u => u.active));   // false
```

3.2.5 Element Insertion & Removal .push(), .pop(), .shift(), .unshift()

push adds an item to the end of an array. pop removes the last item. unshift adds an item to the beginning. shift removes the first item. All four of these change (mutate) the original array directly.

Example Syntax

```
const list = [2, 3];
list.push(4);    // [2, 3, 4]
list.unshift(1); // [1, 2, 3, 4]
list.pop();      // [1, 2, 3]
list.shift();    // [2, 3]
console.log(list); // [2, 3]
```

3.2.6 String Conversion .join(), .toString()

join turns an array into a single string, letting you choose the separator between items. toString does the same thing but always uses a comma as the separator.

Example Syntax

```
const fruits = ['apple', 'banana', 'orange'];
console.log(fruits.join(', ')); // 'apple, banana, orange'
console.log(fruits.toString()); // 'apple,banana,orange'
```

3.3 Object Manipulations

3.3.1 Object and Array Destructuring: Default Values, Aliasing, and Nested Extraction

Destructuring lets you unpack values from an array, or properties from an object, straight into their own variables in a single line. You can give default values in case something is missing, rename a variable using an alias, and even reach into nested objects.

Example Syntax

```
// Object destructuring
const user = { name: 'Alice', age: 25, address: { city: 'Delhi' } };
const { name, age, country = 'India', address: { city } } = user;
console.log(name, age, country, city); // Alice 25 India Delhi

// Renaming (aliasing)
const { name: userName } = user;
console.log(userName); // Alice

// Array destructuring
const colors = ['red', 'green', 'blue'];
const [first, second] = colors;
console.log(first, second); // red green
```

3.3.2 The Spread Operator (...): Shallow Copying and Immutability Safeguards

The spread operator (...) spreads out the elements of an array or the properties of an object. It is the standard way to make a copy without mutating the original value, which is critical when updating state in React.

Keep in mind that spread only creates a SHALLOW copy: nested objects or arrays inside it are still shared between the copy and the original.

Example Syntax

```
const original = { name: 'Alice', age: 25 };
const copy = { ...original, age: 26 }; // creates a NEW object, age updated
console.log(copy); // { name: 'Alice', age: 26 }
console.log(original); // unchanged: { name: 'Alice', age: 25 }

const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];
console.log(arr2); // [1, 2, 3, 4, 5]
```

3.3.3 The Rest Parameter (...): Variable Argument Capture and Partial Extraction

The rest parameter also uses ... but works in the opposite direction from spread: instead of spreading values out, it COLLECTS multiple leftover values into a single array or object. It usually appears in function parameters or in destructuring.

Example Syntax

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4)); // 10

const { name, ...otherDetails } = { name: 'Alice', age: 25, city: 'Delhi' };
```

```
console.log(name);           // Alice
console.log(otherDetails); // { age: 25, city: 'Delhi' }
```

3.4 Structural Copiers Object Manipulations

3.4.1 Creating Objects, Property Access, Adding, Updating, and Deleting Properties, Dynamic Property Names

You can read a property using dot notation or bracket notation. You can add a new property just by assigning to it, update an existing one the same way, and remove one entirely using the delete keyword. Bracket notation also lets you use a variable as the property name, which is called a dynamic property name.

Example Syntax

```
const car = {
  brand: 'Toyota',
  model: 'Corolla',
};

console.log(car.brand); // dot notation
console.log(car['model']); // bracket notation

car.year = 2024; // add a new property
car.model = 'Camry'; // update an existing property
delete car.year; // delete a property

// Dynamic property name
const key = 'color';
const car2 = { [key]: 'red' }; // { color: 'red' }
```

3.4.2 Object Traversal & Built-in Utilities: Object.keys(), Object.values(), Object.entries(), Object.assign(), Object.freeze(), Object.seal(), Object.hasOwn()

Object.keys returns an array of an object's property names. Object.values returns an array of just the values. Object.entries returns an array of [key, value] pairs, which is great for looping. Object.assign copies properties from one or more objects into a target object. Object.freeze completely locks an object so nothing can be changed, added, or removed. Object.seal allows editing existing properties but blocks adding or removing any. Object.hasOwn checks whether a property directly belongs to an object.

Example Syntax

```
const user = { name: 'Alice', age: 25 };

console.log(Object.keys(user)); // ['name', 'age']
console.log(Object.values(user)); // ['Alice', 25]
console.log(Object.entries(user)); // [['name', 'Alice'], ['age', 25]]
```

```
Object.entries(user).forEach(([key, value]) => {
  console.log(`${key}: ${value}`);
});

const merged = Object.assign({}, user, { city: 'Delhi' });
console.log(merged); // { name: 'Alice', age: 25, city: 'Delhi' }

const frozen = Object.freeze({ a: 1 });
frozen.a = 2; // silently fails (throws in strict mode)
console.log(frozen.a); // 1

console.log(Object.hasOwn(user, 'name')); // true
```

3.4.3 Creating & Structuring Arrays of Objects

In real apps, data almost always arrives as an array full of objects, for example a list of students or products. Combining array methods like map, filter, and find with object property access is one of the most common patterns you will use every day.

Example Syntax

```
const students = [
  { id: 1, name: 'Alice', marks: 85 },
  { id: 2, name: 'Bob', marks: 72 },
];

// Common pattern: pull out just one field from every object
const names = students.map(s => s.name);
console.log(names); // ['Alice', 'Bob']

// Common pattern: find one specific object
const topStudent = students.find(s => s.marks > 80);
console.log(topStudent); // { id: 1, name: 'Alice', marks: 85 }
```

4. Numbers, Math, & Dates Methods

4.1 Number Methods

4.1.1 Number.parseInt(), Number.parseFloat(), Number.isNaN(), .toFixed()

parseInt reads a string and pulls out a whole number from the start of it, ignoring anything after. parseFloat does the same but keeps decimal points. Number.isNaN checks strictly whether a value truly is NaN (Not a Number). toFixed rounds a number to a set number of decimal places and returns it as a STRING, which is very handy for showing prices.

Example Syntax

```
console.log(Number.parseInt('42px')); // 42
console.log(Number.parseFloat('3.14kg')); // 3.14
console.log(Number.isNaN('hello' / 2)); // true
console.log((3.14159).toFixed(2)); // '3.14' -> returns a string
```

4.2 Math Methods

4.2.1 .floor(), .ceil(), .round(), .trunc()

Math.floor always rounds a number DOWN. Math.ceil always rounds a number UP. Math.round rounds to the nearest whole number using normal rounding rules. Math.trunc simply chops off the decimal part without any rounding logic at all.

Example Syntax

```
console.log(Math.floor(4.7)); // 4 -> rounds down
console.log(Math.ceil(4.1)); // 5 -> rounds up
console.log(Math.round(4.5)); // 5 -> rounds to nearest
console.log(Math.trunc(4.9)); // 4 -> just removes the decimal part
```

4.2.2 Comparative Selection: .min(), .max(), and Randomization Patterns (.random())

Math.min and Math.max return the smallest or largest number out of a list of values you give them. Math.random generates a random decimal between 0 (included) and 1 (excluded), which is usually scaled up to create random whole numbers, like simulating a dice roll.

Example Syntax

```
console.log(Math.min(3, 7, 1)); // 1
console.log(Math.max(3, 7, 1)); // 7

console.log(Math.random()); // e.g. 0.3821... (between 0 and 1)

// Random whole number between min and max, both inclusive
function randomInt(min, max) {
```

```
    return Math.floor(Math.random() * (max - min + 1)) + min;
}
console.log(randomInt(1, 6)); // simulate a dice roll, 1 to 6
```

4.3 Temporal Engineering via the Date Methods

4.3.1 Epoch Timestamps, Instantiation, and Timezone Offsets

`new Date()` creates a date object representing right now. You can also build a specific date by passing year, month, and day, though be careful: the month is zero-indexed, so January is 0 and December is 11. `getTime` returns the epoch timestamp, which is the number of milliseconds since January 1st, 1970.

Example Syntax

```
const now = new Date();
console.log(now.getTime()); // epoch timestamp in milliseconds

// Month is zero-indexed! 6 actually means July.
const specificDate = new Date(2026, 6, 20);
console.log(specificDate);

console.log(now.getTimezoneOffset()); // minutes of difference from UTC
```

4.3.2 ISO Formatting, Localization Routines

`toISOString` converts a date into a standard, universal text format that is great for storing dates or sending them to a server. `toLocaleDateString` and `toLocaleString` format a date in a human-friendly way based on a language and region you choose, which is what you show to actual users on screen.

Example Syntax

```
const now = new Date();

console.log(now.toISOString()); // '2026-07-20T10:30:00.000Z'

console.log(now.toLocaleDateString('en-IN')); // '20/7/2026'

console.log(
  now.toLocaleString('en-IN', { dateStyle: 'full', timeStyle: 'short' })
);
```

5. Asynchronous Architecture, API Handling, & The Event Loop

5.1 Async Foundations & The Runtime Infrastructure

5.1.1 The Event Loop Engine: Concurrency Model Architecture

JavaScript itself can only do one thing at a time, on a single thread. Yet it still handles things like network requests and timers without freezing the whole page. The secret is the Event Loop, a system that constantly checks whether the main call stack is empty, and if so, pulls the next waiting task in to run.

Example Syntax

```
// No special syntax here -> this is a conceptual model.  
// The Event Loop coordinates three things:  
// 1) The Call Stack (runs your code)  
// 2) Web APIs (handle timers, network requests in the background)  
// 3) Task Queues (hold finished callbacks until the stack is free)
```

5.1.2 Call Stack vs. Web APIs Background Execution Framework

The Call Stack executes your JavaScript code line by line, function by function. When you call something like `setTimeout` or `fetch`, that task is handed off to the browser's Web APIs to run separately in the background, which frees up the Call Stack to keep going with the rest of your code.

Example Syntax

```
console.log('Step 1');  
  
setTimeout(() => {  
  console.log('Step 3 -> handled by a Web API, then queued');  
}, 0);  
  
console.log('Step 2');  
  
// Output order: Step 1, Step 2, Step 3  
// Even with a 0ms delay, setTimeout always waits for the  
// Call Stack to finish everything else first.
```

5.1.3 Microtask Queue (Promises) vs. Macrotask Queue (setTimeout) Priority Routing

When a background task finishes, its callback does not run right away, it waits in a queue. Promise callbacks go into the Microtask Queue. `setTimeout` callbacks go into the Macrotask Queue. The Event Loop always fully empties the Microtask Queue before it picks even one task from the Macrotask Queue, which is why Promises tend to run before a `setTimeout`, even one with a 0ms delay.

Example Syntax

```
console.log('1: Start');
```

```
setTimeout(() => console.log('2: Macrotask (setTimeout)'), 0);

Promise.resolve().then(() => console.log('3: Microtask (Promise)'));

console.log('4: End');

// Output order: 1, 4, 3, 2
// Synchronous code always runs first, then all microtasks,
// and only then does the next macrotask get a turn.
```

5.2 Promise State Topologies

5.2.1 Lifecycle Phases: Pending, Fulfilled, Rejected

Every Promise begins in the pending state, meaning it is still working. It then settles into either fulfilled (success) or rejected (failure), and once it settles it can never change state again.

Example Syntax

```
const promise = new Promise((resolve, reject) => {
  const success = true;
  setTimeout(() => {
    if (success) resolve('Data loaded!');
    else reject('Something went wrong');
  }, 1000);
});

console.log(promise); // Promise { <pending> } at first
```

5.2.2 Execution Control: .then(), .catch(), .finally()

then runs when the Promise is fulfilled. catch runs when the Promise is rejected. finally always runs at the end, no matter what happened, which makes it perfect for cleanup tasks like hiding a loading spinner.

Example Syntax

```
promise
  .then(result => console.log(result))           // runs if resolved
  .catch(error => console.log(error))           // runs if rejected
  .finally(() => console.log('Done, either way')); // always runs
```

5.2.3 Concurrent Aggregation Engines: Promise.all(), Promise.allSettled(), Promise.race()

Promise.all waits for every promise to succeed together, but if even one of them fails, the whole thing immediately fails. Promise.allSettled waits for every promise to finish, whether it succeeded or failed,

and gives you a full report on each one without failing early. `Promise.race` returns as soon as the very FIRST promise settles, whether that is a success or a failure.

Example Syntax

```
const p1 = Promise.resolve('First');
const p2 = Promise.resolve('Second');
const p3 = Promise.reject('Failed');

Promise.all([p1, p2]).then(results => console.log(results));
// ['First', 'Second']

Promise.allSettled([p1, p3]).then(results => console.log(results));
// [ { status: 'fulfilled', value: 'First' },
//   { status: 'rejected', reason: 'Failed' } ]

Promise.race([p1, p2]).then(result => console.log(result));
// 'First' -> whichever one settles first wins
```

5.3 Streamlined Async Control Flows

5.3.1 async/await Syntactic Sugar Engine Mechanics

`async/await` is a friendlier way to write Promise-based code so it reads from top to bottom, almost like normal step-by-step code. Marking a function `async` makes it always return a Promise, and `await` pauses that function until the Promise it is waiting on settles.

Example Syntax

```
async function getData() {
  const result = await promise; // pauses here until the promise settles
  console.log(result);
  return result;
}

getData();
```

5.3.2 Robust Network Exception Management via try-catch Blocks

Wrapping an `await` call inside a `try-catch` block lets you cleanly handle errors from asynchronous code, the same way you would with regular synchronous code. The `finally` block still runs afterward, regardless of success or failure.

Example Syntax

```
async function fetchUser() {
  try {
    const response = await fetch('https://api.example.com/user');
    const data = await response.json();
  }
}
```

```
    console.log(data);  
  } catch (error) {  
    console.log('Something went wrong:', error.message);  
  } finally {  
    console.log('Request finished');  
  }  
}
```

5.3.3 HTTP Data Ingestion: Fetch API Pipeline, Response Validation, and Parsing Strategies

fetch is the browser's built-in method for making HTTP requests to an API. It returns a Promise that resolves with a Response object, and you must then call .json() (which is also a Promise) to get the actual data. Always check response.ok before trusting the data, because fetch does not automatically throw an error for things like a 404 or 500 status code.

Example Syntax

```
async function getUsers() {  
  try {  
    const response = await fetch('https://jsonplaceholder.typicode.com/users');  
    if (!response.ok) {  
      throw new Error(`HTTP error! status: ${response.status}`);  
    }  
    const users = await response.json();  
    console.log(users);  
  } catch (error) {  
    console.log('Fetch failed:', error.message);  
  }  
}
```

6. Browser Storage APIs & DOM Architecture

6.1 Client-Side Persistent Engines

6.1.1 localStorage vs. sessionStorage: Domain Scopes and Retention Lifecycles

localStorage saves data in the browser with no expiry date at all, it stays even after you close the browser or restart the computer. sessionStorage only lasts for the current browser tab, and it disappears the moment that tab is closed. Both of them only store plain strings and are scoped to a single website's domain.

Example Syntax

```
localStorage.setItem('username', 'Alice');
console.log(localStorage.getItem('username')); // 'Alice'
localStorage.removeItem('username');
localStorage.clear(); // removes everything saved

sessionStorage.setItem('tempToken', 'xyz123');
```

6.1.2 Serialization Mechanisms: Data Transformation with JSON.stringify() and JSON.parse()

Since storage only accepts plain strings, you first need to convert an object or array into a string using JSON.stringify before saving it. When reading it back, you convert that string back into a real object using JSON.parse.

Example Syntax

```
const user = { name: 'Alice', age: 25 };
localStorage.setItem('user', JSON.stringify(user)); // save as a string

const savedUser = JSON.parse(localStorage.getItem('user')); // back to an object
console.log(savedUser.name); // 'Alice'
```

6.2 Document Object Model (DOM) Mechanics

6.2.1 The Browser Node Tree Structure and Rendering Layout Pipeline

The DOM represents your entire HTML page as a tree made of individual nodes, one per element. The browser reads your HTML to build this tree, calculates each element's style and position (called layout), and finally paints the actual pixels on screen. Understanding this pipeline explains why making too many rapid DOM changes can slow a page down.

Example Syntax

```
// Conceptual example of the DOM tree for a simple page:
// <html>
//   <body>
```

```
//      <h1>Title</h1>
//      <p>Some text</p>
//    </body>
//  </html>
//
// Each tag above becomes one 'node' in the tree that the
// browser reads, styles, and paints on the screen.
```

6.2.2 Imperative Element Interception and Mutation Patterns

JavaScript can directly select existing elements on the page and change their text, styles, or classes, and it can also create brand new elements and insert them into the page.

Example Syntax

```
const heading = document.querySelector('h1'); // select an element
heading.textContent = 'New Title';             // change its text
heading.style.color = 'blue';                 // change its style
heading.classList.add('highlight');           // add a CSS class

const newPara = document.createElement('p');  // create a new element
newPara.textContent = 'I am new!';
document.body.appendChild(newPara);           // add it to the page
```

6.2.3 Event Delegation, Event Bubbling, Capturing, and Performance Optimization

When you click an element, the event first travels DOWN from the top of the page to your target element, called the capturing phase, then travels BACK UP from the target to the top, called the bubbling phase. Event Delegation takes advantage of bubbling by placing just ONE listener on a parent element instead of many listeners on every child, which is far more efficient, especially for long lists.

Example Syntax

```
// Instead of adding a listener to every single <li>,
// add ONE listener to the parent <ul>.
document.querySelector('ul').addEventListener('click', function (event) {
  if (event.target.tagName === 'LI') {
    console.log('You clicked:', event.target.textContent);
  }
});

// To listen during the capturing phase instead of bubbling:
element.addEventListener('click', handler, { capture: true });
```

8. TypeScript Integration & Methods

8.1 Type System Foundations

8.1.1 Primitives (string, number, boolean, null, undefined) vs. any/unknown

TypeScript adds 'types' on top of JavaScript so that mistakes get caught before your code ever runs. Primitive types describe simple, basic values. The any type turns off type-checking completely for that value, which is risky and should usually be avoided. The unknown type is a safer alternative: it also accepts any value, but it forces you to check what it actually is before you can use it.

Example Syntax

```
let username: string = 'Alice';
let age: number = 25;
let isActive: boolean = true;
let empty: null = null;
let notAssigned: undefined = undefined;

let flexible: any = 'hello';
flexible = 42; // no error, but this defeats the whole purpose of TypeScript

let safer: unknown = 'hello';
// safer.toUpperCase(); // Error -> must check the type first
if (typeof safer === 'string') {
  console.log(safer.toUpperCase()); // Works after the check
}
```

8.1.2 Structure Typing: Defining Contracts via type Aliases vs. interface Systems

Both type and interface let you describe the shape of an object, meaning what properties it must have. interface is generally preferred for objects and classes because it can be extended and even merged with itself. type is more flexible, because it can also describe unions, primitives, and tuples, which interface cannot do.

Example Syntax

```
// Using interface
interface User {
  name: string;
  age: number;
}

// Using type
type UserType = {
  name: string;
  age: number;
};
```

```
const user: User = { name: 'Alice', age: 25 };

// interface can be extended:
interface Admin extends User {
  role: string;
}
```

8.1.3 Flexible Types: Union Types, Intersection Types

A union type, written with |, means a value is allowed to be ONE of several listed types. An intersection type, written with &, COMBINES multiple types into one, meaning the value has to satisfy ALL of them at the same time.

Example Syntax

```
// Union type
let id: string | number;
id = 'abc123'; // valid
id = 123;      // valid

// Intersection type
type Person = { name: string };
type Employee = { salary: number };
type Staff = Person & Employee; // must have BOTH name AND salary

const staff: Staff = { name: 'Alice', salary: 50000 };
```

8.2 TypeScript Functional & Transformation Methods

8.2.1 Strongly Typing Function Arguments, Return Structures, and Async Promises

You can label the type of every function parameter as well as what type the function returns. For async functions, the return type is always wrapped inside Promise, because an async function always hands back a Promise.

Example Syntax

```
function add(a: number, b: number): number {
  return a + b;
}

// Async function returning a Promise of a specific shape
async function fetchUser(): Promise<{ name: string; age: number }> {
  const response = await fetch('/api/user');
  return response.json();
}
```

8.2.2 Generics (<T>): Creating Reusable, Type-Safe Data Contracts

Generics let you write one function or type that works correctly with MANY different types, while TypeScript still keeps everything type-safe behind the scenes. The <T> you see is just a placeholder standing in for whatever type actually gets passed in.

Example Syntax

```
function getItem<T>(items: T[]): T {
    return items[0];
}

console.log(getItem<number>([1, 2, 3])); // 1
console.log(getItem<string>(['a', 'b', 'c'])); // 'a'

// Generic interface example
interface ApiResponse<T> {
    data: T;
    success: boolean;
}
const response: ApiResponse<string> = { data: 'Hello', success: true };
```

8.2.3 Advanced Utility Mappings: Partial<T>, Omit<T, K>, Pick<T, K>, and Readonly<T>

These are built-in TypeScript helpers that transform an existing type into a new one. Partial makes every property OPTIONAL. Pick keeps only the properties you name. Omit removes only the properties you name, keeping everything else. Readonly makes every property unchangeable once the object is created.

Example Syntax

```
interface User {
    id: number;
    name: string;
    email: string;
}

type PartialUser = Partial<User>; // every property optional
type UserPreview = Pick<User, 'id' | 'name'>; // only id and name
type UserWithoutEmail = Omit<User, 'email'>; // everything except email
type ReadonlyUser = Readonly<User>; // cannot change any property

const updates: PartialUser = { name: 'New Name' }; // valid, other fields optional
```

8.3 Object-Oriented Programming (OOP)

8.3.1 Class Fundamentals: Class Declaration, Object Creation, Constructors, Properties, Methods

A class is a blueprint for creating objects that share the same properties and methods. The constructor is a special method that runs automatically whenever you create a new object with the new keyword, and it is typically used to set up the initial values of properties.

Example Syntax

```
class Car {
  brand: string;
  model: string;

  constructor(brand: string, model: string) {
    this.brand = brand;
    this.model = model;
  }

  displayInfo(): string {
    return `${this.brand} ${this.model}`;
  }
}

const myCar = new Car('Toyota', 'Corolla');
console.log(myCar.displayInfo()); // 'Toyota Corolla'
```

8.3.2 Access Modifiers: public, private, protected, readonly

public is the default, and it means a property or method can be accessed from anywhere. private means it can only be accessed from inside that exact class. protected means it can be accessed inside that class AND any class that extends (inherits from) it. readonly means the property can only be set once, usually inside the constructor, and never changed again.

Example Syntax

```
class BankAccount {
  public accountHolder: string;
  private balance: number;
  protected accountType: string = 'Savings';
  readonly accountNumber: string;

  constructor(holder: string, balance: number, accNum: string) {
    this.accountHolder = holder;
    this.balance = balance;
    this.accountNumber = accNum;
  }
}

const account = new BankAccount('Alice', 1000, 'AC123');
console.log(account.accountHolder); // works fine -> it is public
// console.log(account.balance);    // Error -> balance is private
```

8.3.3 Encapsulation: Data Hiding, Getter Methods, Setter Methods, Access Control

Encapsulation means hiding a property's raw data (by marking it private) and only allowing controlled access to it through special getter and setter methods. This lets you add validation rules whenever the data is read or changed, protecting it from invalid values.

Example Syntax

```
class Person {
  private _age: number = 0;

  get age(): number {
    return this._age;
  }

  set age(value: number) {
    if (value < 0) {
      throw new Error('Age cannot be negative');
    }
    this._age = value;
  }
}

const p = new Person();
p.age = 25;           // calls the setter
console.log(p.age);  // calls the getter -> 25
```

8.3.4 Inheritance: extends, Base Class, Derived Class, Method Overriding, super

Inheritance lets a child class (the derived class) reuse and build on top of a parent class (the base class), using the extends keyword. Method overriding lets the child class replace a parent's method with its own version. The super keyword is used to call the parent class's constructor or its original methods.

Example Syntax

```
class Animal {
  constructor(public name: string) {}
  makeSound(): string {
    return 'Some generic sound';
  }
}

class Dog extends Animal {
  makeSound(): string { // method overriding
    return `${this.name} says Woof!`;
  }
}

const dog = new Dog('Rex');
console.log(dog.makeSound()); // 'Rex says Woof!'
```

8.3.5 Polymorphism: Runtime Polymorphism, Method Overriding

Polymorphism literally means 'many forms'. Different classes can each implement a method with the exact same name, but with completely different behavior. At runtime, JavaScript automatically calls the correct version based on the actual type of the object.

Example Syntax

```
class Cat extends Animal {
  makeSound(): string {
    return `${this.name} says Meow!`;
  }
}

const animals: Animal[] = [new Dog('Rex'), new Cat('Whiskers')];
animals.forEach(animal => console.log(animal.makeSound()));
// 'Rex says Woof!'
// 'Whiskers says Meow!'
// Same method call, different behavior per class -> that is polymorphism
```

8.3.6 Abstraction: Abstract Classes, Abstract Methods

An abstract class is a blueprint that can never be directly turned into an object with new; it only exists to be extended. An abstract method has no body at all in the base class, forcing every child class to provide its own implementation of it.

Example Syntax

```
abstract class Shape {
  abstract calculateArea(): number; // no body -> children must implement it

  describe(): string {
    return `This shape has an area of ${this.calculateArea()}`;
  }
}

class Circle extends Shape {
  constructor(private radius: number) {
    super();
  }
  calculateArea(): number {
    return Math.PI * this.radius * this.radius;
  }
}

const circle = new Circle(5);
console.log(circle.describe());
// const shape = new Shape(); // Error -> cannot create an abstract class directly
```

8.3.7 Interfaces in OOP: Implementing Interfaces, Multiple Interface Implementation, Interface Extension

An interface defines a contract listing exactly which properties and methods a class must have, without providing any implementation itself. A single class is allowed to implement multiple interfaces at once, and interfaces themselves can extend one another.

Example Syntax

```
interface Flyable {
  fly(): void;
}
interface Swimmable {
  swim(): void;
}

// A class implementing multiple interfaces
class Duck implements Flyable, Swimmable {
  fly(): void {
    console.log('Duck is flying');
  }
  swim(): void {
    console.log('Duck is swimming');
  }
}

// Interface extension
interface AdvancedFlyable extends Flyable {
  maxAltitude: number;
}
```

8.3.8 Static Members: Static Properties, Static Methods

Members marked static belong to the CLASS itself, not to any individual object created from it. You access them directly through the class name, without ever needing to write new.

Example Syntax

```
class MathHelper {
  static PI: number = 3.14159;

  static square(n: number): number {
    return n * n;
  }
}

console.log(MathHelper.PI);           // 3.14159 -> no instance needed
console.log(MathHelper.square(5));    // 25
```